

PROJET DE FIN DE MODULE

Module : Deep Learning

**Conception, implémentation, comparaison et analyse critique
de modèles de deep learning pour données tabulaires, images et
séquences (MLP • CNN • RNN • LSTM • GRU)**

Réalisé par : MAFROUZ Maryem

Encadré par : Mme. HIDILA Zineb

Filière : 4IADM-G3

Année universitaire 2025–2026

Table des matières

Table des matières	2
1. Introduction	4
2. Objectifs généraux	4
3. Partie I — MLP et ingénierie PyTorch	5
3.1 Présentation du dataset.....	5
3.2 Fondements théoriques.....	5
3.3 Préparation des données	5
3.4 Architectures implémentées.....	5
3.5 Résultats expérimentaux	6
3.5.1 Comparaison des stratégies d'initialisation	6
3.5.2 Comparaison des architectures Sequential vs classe personnalisée	6
3.5.3 Évaluation finale détaillée (modèle rechargé)	6
3.6 Analyse critique.....	7
3.7 Question de synthèse — Partie I.....	7
4. Partie II — CNN et vision par ordinateur	8
4.1 Présentation du dataset.....	8
4.2 Pourquoi un MLP est peu adapté aux images	8
4.3 Calculs dimensionnels	8
4.4 Implémentations manuelles vs PyTorch	8
4.5 Architecture CNN implémentée.....	8
4.6 Résultats expérimentaux	9
4.6.1 Étude de l'influence des choix architecturaux	9
4.6.2 Accuracy par classe	9
4.6.3 Visualisation des cartes de caractéristiques	9
4.7 Analyse critique.....	9
4.8 Question de synthèse — Partie II.....	10
5. Partie III — RNN, LSTM, GRU et Seq2Seq	11
5.1 Présentation du dataset.....	11
5.2 Modèle de langage et perplexité	11
5.3 Préparation des données	11
5.4 Implémentation et comparaison RNN / LSTM / GRU	11
5.5 BPTT et gradient clipping	12
5.6 Architecture Seq2Seq (encodeur–décodeur)	12

5.7 Stratégies de décodage : glouton vs beam search.....	12
5.8 Évaluation : perplexité et BLEU.....	13
5.9 Tableau de synthèse théorique	13
5.10 Analyse critique.....	13
5.11 Question de synthèse — Partie III.....	13
6. Discussion scientifique transversale	15
6.1 Données tabulaires — absence de structure géométrique exploitable	15
6.2 Données image — structure spatiale locale et hiérarchique.....	15
6.3 Données séquentielles — dépendance temporelle et ordre.....	15
6.4 Synthèse.....	15
7. Conclusion générale.....	16
8. Limites globales et perspectives	16
9. Annexe expérimentale	16

1. Introduction

Ce rapport constitue l'évaluation finale du module Deep Learning. Conformément au cahier des charges, il couvre trois familles d'architectures appliquées à trois natures de données distinctes : un perceptron multicouche (MLP) pour des données tabulaires de santé (Heart Disease), un réseau de neurones convolutionnel (CNN) pour la classification d'images (CIFAR-10), et des architectures récurrentes (RNN, LSTM, GRU, Seq2Seq) pour l'analyse de séquences textuelles (IMDb). L'objectif transversal est de montrer comment le choix d'une architecture découle de la structure intrinsèque des données — absence de structure spatiale ou temporelle pour le tabulaire, structure spatiale locale pour l'image, dépendance séquentielle et temporelle pour le texte.

Pour chaque partie, la démarche suit le même canevas : étude théorique, implémentation sous PyTorch, étude expérimentale comparative, analyse critique des résultats, puis réponse argumentée à la question de synthèse portant sur le dataset réel utilisé.

2. Objectifs généraux

- Maîtriser la construction de modèles avec `nn.Module` (versions Sequential et classe personnalisée).
- Comprendre et implémenter manuellement les opérations fondamentales (corrélacion croisée, pooling) puis les comparer à PyTorch.
- Conduire des expériences comparatives rigoureuses (initialisation, architecture, hyperparamètres).
- Évaluer les modèles avec des métriques adaptées à chaque tâche (accuracy, F1, perplexité, BLEU).
- Relier théorie, implémentation et résultats expérimentaux dans une analyse critique.

3. Partie I — MLP et ingénierie PyTorch

3.1 Présentation du dataset

Le dataset utilisé est Heart Disease (heart.csv), un jeu de données tabulaire de classification binaire en cardiologie comportant 1 025 observations et 14 variables (âge, sexe, type de douleur thoracique, pression artérielle, cholestérol, fréquence cardiaque maximale, etc.), la variable cible target indiquant la présence (1) ou l'absence (0) de maladie cardiaque. L'exploration n'a révélé aucune valeur manquante. La répartition des classes est globalement équilibrée, ce qui justifie l'usage direct de l'accuracy comme métrique principale, complétée par la precision, le recall et le F1-score.

3.2 Fondements théoriques

Un MLP est un empilement de couches denses (fully connected) séparées par des non-linéarités. Sous PyTorch, deux approches permettent de le construire :

- `nn.Sequential` : empilement déclaratif des couches dans l'ordre, rapide à écrire mais peu flexible pour des architectures non linéaires.
- Une classe personnalisée héritant de `nn.Module` : la méthode `__init__` déclare les couches, et `forward()` définit explicitement le flux de données, ce qui offre davantage de contrôle (branches, skip-connections, etc.).

Chaque paramètre du modèle (poids et biais) est un tenseur dont l'attribut `requires_grad=True` est activé automatiquement ; PyTorch construit alors dynamiquement un graphe de calcul lors du `forward`, et la méthode `.backward()` applique la règle de dérivation en chaîne pour calculer le gradient de la loss par rapport à chaque paramètre (rétropropagation). L'optimiseur (ici Adam) met ensuite à jour les poids selon ce gradient. La méthode `named_parameters()` permet d'inspecter nom, forme et valeur de chaque paramètre, tandis que `state_dict()` fournit un dictionnaire sérialisable de tous les paramètres et statistiques (utile pour la sauvegarde et le rechargement). Le device (CPU/GPU) détermine où les tenseurs et les calculs sont physiquement exécutés ; modèle et données doivent impérativement résider sur le même device.

3.3 Préparation des données

1. Séparation features/cible (X, y).
2. Split stratifié 70 % train / 15 % validation / 15 % test (stratify=y pour préserver la proportion des classes).
3. Normalisation des variables continues avec `StandardScaler`, ajusté uniquement sur le train puis appliqué à validation et test (évite la fuite d'information).
4. Conversion en tenseurs PyTorch (float32 pour X, long pour y) puis transfert sur le device disponible.

3.4 Architectures implémentées

Les deux versions du MLP partagent la même architecture : couche d'entrée (13 features) → Linear(128) → BatchNorm1d → ReLU → Dropout(0.3) → Linear(64) → BatchNorm1d → ReLU → Dropout(0.3) → Linear(2). Le BatchNorm stabilise l'entraînement en normalisant les activations intermédiaires, et le Dropout limite le surapprentissage en désactivant aléatoirement une fraction des neurones à chaque pas.

3.5 Résultats expérimentaux

3.5.1 Comparaison des stratégies d'initialisation

Trois stratégies d'initialisation des poids ont été testées sur 50 époques : gaussienne ($\sigma = 0,02$), constante (0,5) et Xavier uniforme.

Initialisation	Accuracy (test)
Gaussienne	91,56 %
Constante	58,44 %
Xavier	87,01 %

Tableau 1 — Accuracy selon la stratégie d'initialisation des poids.

L'initialisation constante effondre nettement les performances (58,44 %, proche du hasard) : tous les neurones d'une même couche démarrent avec des poids identiques, donc des gradients identiques, et restent symétriques tout au long de l'entraînement (problème de symétrie). Les initialisations gaussienne et Xavier brisent cette symétrie et permettent un apprentissage correct, avec un léger avantage pour la gaussienne dans cette configuration.

3.5.2 Comparaison des architectures Sequential vs classe personnalisée

Modèle	Accuracy (test)
nn.Sequential	87,66 %
Classe personnalisée	83,77 %

Tableau 2 — Comparaison des deux modes de construction du MLP.

Les deux implémentations étant strictement équivalentes sur le plan architectural, l'écart observé (≈ 4 points) s'explique principalement par la stochasticité de l'initialisation et de l'optimisation (mini-batches, dropout), et non par une différence structurelle réelle entre les deux approches de construction.

3.5.3 Évaluation finale détaillée (modèle rechargé)

Métrique	Valeur
Accuracy	83,77 %
Precision	82,14 %
Recall	87,34 %
F1-score	84,66 %

Tableau 3 — Métriques du modèle rechargé depuis `best_model.pth` sur le jeu de test.

Le rechargement via `load_state_dict()` reproduit fidèlement les performances du modèle sauvegardé, confirmant la cohérence du pipeline de sauvegarde/chargement et de la gestion du device.

3.6 Analyse critique

Les courbes de loss training/validation décroissent de manière conjointe et régulière sur 50 époques, sans divergence marquée, ce qui indique une absence de surapprentissage sévère, en partie grâce au Dropout et au BatchNorm. Le recall (87,3 %) supérieur à la precision (82,1 %) signifie que le modèle rate peu de cas positifs (malades), ce qui est préférable dans un contexte médical où un faux négatif est plus coûteux qu'un faux positif. Les principales limites observées sont : (i) la sensibilité du résultat à l'initialisation et au seed, qui rend nécessaire une moyenne sur plusieurs runs pour une comparaison rigoureuse ; (ii) la petite taille du dataset (1 025 lignes), qui limite la capacité du modèle à généraliser sur un MLP relativement profond ; (iii) l'absence de validation croisée, qui aurait renforcé la robustesse statistique des conclusions.

3.7 Question de synthèse — Partie I

« Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset réel, et quelles sont ses principales limites au regard de la structure statistique des données étudiées ? »

Sur le dataset Heart Disease, un MLP correctement régularisé (BatchNorm + Dropout) et bien initialisé atteint une accuracy de l'ordre de 84 à 92 %, ce qui est compétitif pour ce type de données. Les données tabulaires n'ont pas de structure spatiale ou séquentielle exploitable : chaque variable est a priori indépendante des autres en position, ce qui rend les couches denses, capables d'apprendre n'importe quelle combinaison linéaire puis non linéaire des features, naturellement adaptées. Le MLP excelle ici car les interactions entre variables (par exemple cholestérol $\times$ âge) peuvent être capturées par les couches cachées sans hypothèse a priori sur leur structure.

Ses limites restent toutefois réelles : (1) il ignore la nature parfois mixte des variables (catégorielles encodées numériquement vs continues), traitées indistinctement par les couches Linear ; (2) il est sensible au bruit et à la petite taille des datasets tabulaires, contrairement à des méthodes comme les forêts aléatoires ou le gradient boosting, souvent plus robustes et plus interprétables sur ce type de données ; (3) l'absence d'inductive bias adapté (contrairement à la localité des CNN ou à la récurrence des RNN) signifie que le MLP doit apprendre toute structure depuis zéro à partir d'un nombre limité d'exemples, ce qui peut le pénaliser face à des algorithmes à base d'arbres sur des jeux de données tabulaires de taille modeste.

4. Partie II — CNN et vision par ordinateur

4.1 Présentation du dataset

Le dataset utilisé est CIFAR-10 : 60 000 images couleur 32×32 réparties en 10 classes (avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, navire, camion), avec 50 000 images d'entraînement et 10 000 de test. Les pixels sont normalisés dans [0, 1], la distribution des classes est équilibrée (6 000 images par classe) et aucune valeur manquante n'est présente.

4.2 Pourquoi un MLP est peu adapté aux images

Un MLP appliqué à une image la traite comme un vecteur plat de pixels (ici $3 \times 32 \times 32 = 3\,072$ valeurs), ce qui détruit immédiatement la structure spatiale 2D : deux pixels voisins dans l'image n'ont plus aucune proximité particulière dans le vecteur d'entrée. De plus, le nombre de paramètres explose : une seule couche cachée de 512 neurones nécessite déjà $3\,072 \times 512 \approx 1,57$ million de paramètres, ce qui favorise le surapprentissage et alourdit l'entraînement. Les CNN répondent à ces limites par trois principes fondateurs :

- Localité : chaque filtre n'analyse qu'une petite région locale de l'image (champ récepteur), exploitant le fait que l'information pertinente est souvent locale (contours, textures).
- Partage des poids : le même filtre est appliqué sur toute l'image, ce qui réduit drastiquement le nombre de paramètres et rend la détection de motifs invariante à la translation.
- Hiérarchie des représentations : l'empilement de couches convolutionnelles permet de passer progressivement de motifs bas niveau (bords, couleurs) à des concepts plus abstraits (formes, objets).

4.3 Calculs dimensionnels

Taille de sortie d'une convolution : $\text{Output} = (W - F + 2P) / S + 1$, avec W la taille d'entrée, F la taille du filtre, P le padding et S le stride. Exemple : entrée 32×32, filtre 3×3, padding 1, stride 1 $\rightarrow$ Output = $(32 - 3 + 2)/1 + 1 = 32$ (la dimension spatiale est préservée). Taille après pooling : $\text{Output} = (W - F)/S + 1$. Exemple : entrée 32×32, fenêtre 2×2, stride 2 $\rightarrow$ Output = $(32 - 2)/2 + 1 = 16$ (division par deux de chaque dimension).

4.4 Implémentations manuelles vs PyTorch

Les opérations de corrélation croisée 2D, de max-pooling et d'average-pooling ont été codées manuellement en NumPy (double boucle glissante sur l'image), puis comparées aux couches équivalentes de PyTorch (nn.MaxPool2d, nn.AvgPool2d). Sur l'exemple test (image 4×4, fenêtre 2×2, stride 2), le max-pooling manuel et nn.MaxPool2d produisent exactement les mêmes valeurs ([[6,8],[14,16]]), de même que l'average-pooling ([[3.5, 5.5],[11.5, 13.5]]), validant la correction des implémentations manuelles.

4.5 Architecture CNN implémentée

Un CNN inspiré de LeNet mais étendu (trois blocs convolutionnels Conv2d–BatchNorm2d–ReLU avec 32, 64 puis 128 filtres, padding 1, suivis de MaxPooling) a été implémenté, puis comparé à un MLP simple à deux couches (Linear(3072→512)–ReLU–Linear(512→10)) sur la même tâche de classification CIFAR-10.

4.6 Résultats expérimentaux

Modèle	Accuracy test finale	Observation
MLP (2 couches)	≈ 47,1 %	Plafonne rapidement
CNN (LeNet amélioré)	73,86 %	+5 époques seulement

Tableau 4 — Comparaison CNN vs MLP sur CIFAR-10 (5 époques pour le CNN, 10 pour le MLP).

Avec un budget d'entraînement pourtant moindre (5 époques contre 10 pour le MLP), le CNN atteint une accuracy supérieure de plus de 25 points, confirmant empiriquement la pertinence des principes de localité et de partage des poids pour les données image. La courbe de loss du CNN décroît plus rapidement et de manière plus stable que celle du MLP, qui plafonne après quelques époques.

4.6.1 Étude de l'influence des choix architecturaux

Plusieurs variantes ont été testées par rapport au modèle de référence : suppression du padding (padding = 0), augmentation du stride (stride = 2), remplacement du max-pooling par de l'average-pooling, et ajout d'une convolution 1×1. L'absence de padding réduit progressivement la taille spatiale des cartes de caractéristiques à chaque couche, ce qui limite la profondeur exploitable du réseau. Un stride plus élevé accélère le calcul mais réduit la résolution spatiale transmise aux couches suivantes, au prix d'une perte d'information fine. La convolution 1×1 ne modifie pas la dimension spatiale mais permet de mélanger l'information entre canaux et de contrôler le nombre de filtres à moindre coût, une technique reprise des architectures Inception/ResNet.

4.6.2 Accuracy par classe

L'analyse par classe montre que les classes les plus difficiles à reconnaître pour le CNN de référence sont bird (67,6 % d'accuracy) et cat (56,4 %), deux catégories visuellement proches d'autres classes animales (dog, deer) et présentant une grande variabilité de pose et de fond, contre des classes plus discriminantes comme automobile ou ship qui atteignent des taux nettement supérieurs.

4.6.3 Visualisation des cartes de caractéristiques

Un hook a été placé sur la première couche convolutionnelle (conv1) pour visualiser les feature maps produites en sortie. L'interprétation montre que certains filtres détectent des bords horizontaux ou verticaux, d'autres réagissent à des zones de couleur homogène : les filtres de la première couche sont des détecteurs de caractéristiques bas niveau (contours, contrastes), tandis que les couches plus profondes combinent ces éléments en représentations de plus en plus abstraites, conformément au principe de hiérarchie des représentations.

4.7 Analyse critique

Le CNN surpasse nettement le MLP simple sur cette tâche, validant le choix architectural attendu pour des données image. Cependant, l'accuracy de 73,86 % obtenue en seulement 5 époques reste perfectible : un entraînement plus long, l'usage de data augmentation plus poussée (déjà esquissée avec `RandomHorizontalFlip` et `RandomRotation`), ou une architecture plus profonde (type ResNet) permettraient probablement de dépasser 85 à 90 % sur CIFAR-10. Les limites observées concernent surtout la confusion entre classes animales visuellement proches, un problème classique de ce dataset à faible résolution (32×32), où le manque de détails fins limite la capacité de discrimination même d'un CNN bien conçu.

4.8 Question de synthèse — Partie II

« Pourquoi un CNN est-il plus pertinent qu'un MLP pour une tâche de classification d'images sur un dataset réel, et comment les choix de padding, stride, pooling et profondeur influencent-ils réellement les performances du modèle ? »

Les résultats expérimentaux sur CIFAR-10 confirment l'argument théorique : le CNN (73,86 %) surpasse largement le MLP ($\approx 47,1$ %) à budget d'entraînement comparable, car il exploite la structure spatiale de l'image via la localité et le partage des poids, ce qui réduit le nombre de paramètres effectifs et impose un biais inductif adapté à la nature 2D des données — un MLP doit au contraire réapprendre cette structure depuis un vecteur plat, ce qui le pénalise fortement avec un budget de données et d'époques limité.

Le padding contrôle la préservation de la résolution spatiale au fil des couches : sans padding, les cartes de caractéristiques rétrécissent à chaque convolution, ce qui limite la profondeur exploitable du réseau. Le stride agit comme un facteur de sous-échantillonnage : un stride élevé accélère les calculs et réduit la mémoire requise, mais au prix d'une perte de résolution et donc d'information fine, potentiellement préjudiciable pour des classes visuellement proches (bird/cat dans nos résultats). Le pooling (max ou moyenne) introduit une invariance locale aux petites translations et réduit la dimensionnalité progressivement, le max-pooling étant généralement préféré pour préserver les activations les plus discriminantes. Enfin, la profondeur du réseau (nombre de blocs convolutionnels) conditionne le niveau d'abstraction atteint par les représentations apprises, comme l'illustre la visualisation des feature maps de conv1 (motifs bas niveau) face aux couches plus profondes.

5. Partie III — RNN, LSTM, GRU et Seq2Seq

5.1 Présentation du dataset

Le corpus utilisé est IMDb Sentiment, composé de 50 000 critiques de films en anglais, équitablement réparties entre sentiments positif et négatif (25 000 chacun). La longueur moyenne des critiques est d'environ 231 tokens (min 4, max 2 470, écart-type 171), ce qui justifie une troncature/padding à une longueur maximale fixe pour la mise en mini-lots.

5.2 Modèle de langage et perplexité

Un modèle de langage estime la probabilité d'une séquence de tokens en la factorisant par la règle de chaîne des probabilités : $P(w_1, \dots, w_T) = \prod_t P(w_t \mid w_1, \dots, w_{t-1})$. Chaque mot est ainsi prédit conditionnellement à son contexte gauche. La perplexité, définie comme l'exponentielle de la cross-entropie moyenne ($PPL = \exp(CE)$), mesure le degré d'incertitude du modèle : une perplexité de 2,7 correspond à une cross-entropie de 1 bit naturel, et plus la perplexité est faible, plus le modèle prédit le mot suivant avec confiance (une perplexité de 1 correspondrait à une prédiction parfaite).

5.3 Préparation des données

- Nettoyage : suppression des balises HTML, conservation des lettres uniquement, mise en minuscule.
- Tokenisation simple par espaces, construction d'un vocabulaire de 10 000 mots avec tokens spéciaux PAD=0, UNK=1, BOS=2, EOS=3.
- Encodage des séquences en identifiants, troncature à une longueur maximale, padding pour homogénéiser les longueurs au sein d'un mini-lot.
- Split train/val/test : 40 000 / 5 000 / 5 000 critiques, soit 625 mini-lots d'entraînement.
- Un masquage des positions de padding est appliqué via les longueurs réelles transmises au `pack_padded_sequence/collate_fn`, afin que le RNN ignore les pas de temps artificiels.

5.4 Implémentation et comparaison RNN / LSTM / GRU

Un module générique `SentimentModel` a été implémenté, paramétrable par le type de cellule récurrente ('RNN', 'LSTM' ou 'GRU'). Les trois architectures ont été entraînées 5 époques dans des conditions identiques (mêmes hyperparamètres, même optimiseur Adam) sur la tâche de classification de sentiment.

Modèle	Test Accuracy	Temps d'entraînement	Nombre de paramètres
RNN simple	69,32 %	68,0 s	1 346 306
GRU	88,54 %	72,9 s	1 478 402
LSTM	87,60 %	73,9 s	1 544 450

Tableau 5 — Comparaison RNN / GRU / LSTM sur la classification de sentiment IMDb.

Le RNN simple plafonne nettement en dessous des architectures à portes (69,3 % contre 87,6–88,5 %), illustrant concrètement le problème de disparition du gradient (vanishing gradient) sur des séquences

longues (jusqu'à 200 tokens) : le RNN simple peine à conserver l'information utile sur de longues distances. GRU et LSTM, grâce à leurs mécanismes de portes (gating) qui régulent explicitement le flux d'information dans l'état caché, mémorisent bien mieux le contexte, pour un coût de calcul à peine supérieur (+5 à 6 s) et un nombre de paramètres comparable. GRU obtient ici une légère avance sur LSTM (88,54 % vs 87,60 %) tout en étant plus léger (moins de portes : 3 contre 4), ce qui en fait souvent un bon compromis performance/coût.

5.5 BPTT et gradient clipping

L'entraînement des RNN repose sur la rétropropagation à travers le temps (BPTT) : le réseau est déroulé sur tous les pas de temps de la séquence, et le gradient de la loss est rétropropagé à travers cette chaîne de dépendances temporelles, ce qui peut conduire à une explosion ou une disparition du gradient selon la longueur de la séquence et les valeurs propres des matrices de poids récurrentes. Une mesure expérimentale de la norme du gradient avec et sans clipping (sur un LSTM entraîné avec SGD à $\text{lr}=0,1$, configuration choisie pour amplifier une éventuelle instabilité) a été réalisée sur 30 pas. Dans cette expérience, la norme maximale observée reste identique avec et sans clipping (0,274), ce qui indique qu'aucune explosion de gradient significative ne s'est produite dans cette configuration particulière — le clipping agit alors comme un filet de sécurité qui n'intervient que lorsque la norme dépasse effectivement le seuil fixé, sans modifier l'entraînement lorsque ce n'est pas le cas.

5.6 Architecture Seq2Seq (encodeur–décodeur)

Un système Seq2Seq avec encodeur et décodeur LSTM a été construit pour une tâche de modèle de langage génératif (prédiction du prochain token), entraîné avec teacher forcing (le décodeur reçoit en entrée, à l'entraînement, le token de référence plutôt que sa propre prédiction précédente, ce qui stabilise et accélère l'apprentissage). Le modèle comporte environ 4,38 millions de paramètres.

Époque	Train Loss / PPL	Val Loss / PPL
1	6,45 / 631,0	6,31 / 550,1
3	5,98 / 397,3	6,24 / 512,8
5	5,84 / 344,0	6,18 / 484,1

Tableau 6 — Évolution de la loss et de la perplexité du Seq2Seq (sélection d'époques).

La perplexité d'entraînement diminue régulièrement (631 $\rightarrow$ 344), tandis que la perplexité de validation se stabilise autour de 484–550 après quelques époques, signe d'un début de surapprentissage : le modèle mémorise progressivement le train sans gain équivalent en généralisation, ce qui est cohérent avec un entraînement court (5 époques) sur une tâche de génération libre, intrinsèquement plus difficile que la classification binaire de sentiment.

5.7 Stratégies de décodage : glouton vs beam search

Deux stratégies de décodage ont été comparées sur des exemples du jeu de test : le décodage glouton (greedy), qui sélectionne à chaque pas le token de plus forte probabilité, et le beam search ($k = 3$), qui

maintient les 3 séquences partielles les plus probables à chaque étape avant de choisir la meilleure séquence complète. Les générations obtenues restent répétitives et peu cohérentes sur le plan sémantique (par exemple des suites de tokens comme « i was this movie i i i i »), ce qui reflète le faible entraînement (5 époques) et la perplexité encore élevée du modèle de langage plutôt qu'une limite intrinsèque de l'architecture Seq2Seq. Le beam search produit des séquences légèrement plus variées que le glouton (introduction du token <UNK> à différents endroits) mais ne résout pas le problème de fond lié à l'insuffisance de l'entraînement.

5.8 Évaluation : perplexité et BLEU

La perplexité finale du Seq2Seq sur le test est de 485,07, cohérente avec la perplexité de validation observée en fin d'entraînement. Les scores BLEU calculés sur 5 exemples sont nuls (0 % pour le glouton comme pour le beam search), ce qui s'explique par le faible nombre d'époques d'entraînement (5) au regard de la difficulté de la tâche de génération libre sur un vocabulaire de 10 000 mots : avec davantage d'époques et un corpus mieux ciblé pour une tâche de traduction ou de résumé (plutôt qu'un modèle de langage générique sur IMDb), des scores BLEU non nuls seraient attendus.

5.9 Tableau de synthèse théorique

Architecture	Formule simplifiée	Mémoire / gradient	Coût	Accuracy obtenue
RNN simple	$\tanh(Wx \cdot x + Wh \cdot h)$	Faible (vanishing)	Rapide	≈ 69 %
LSTM	4 portes (f, i, g, o)	Bonne (cell state)	Plus lent	≈ 88 %
GRU	3 portes (r, z, $\tilde{h}$)	Bonne, plus légère	Intermédiaire	≈ 89 %

Tableau 7 — Synthèse comparative des architectures récurrentes étudiées.

5.10 Analyse critique

Les résultats confirment l'intérêt des mécanismes de portes (GRU/LSTM) par rapport au RNN simple pour une tâche de classification sur séquences longues. En revanche, la partie génération (Seq2Seq) souffre d'un sous-entraînement manifeste (perplexité élevée, BLEU nul), ce qui est attendu compte tenu du nombre réduit d'époques (5) imposé par les contraintes de calcul du projet, mais qui limite la portée des conclusions sur la qualité réelle du décodage. Une perspective d'amélioration consisterait à entraîner sur davantage d'époques, à utiliser un mécanisme d'attention (Bahdanau/Luong) pour pallier la limitation du vecteur de contexte fixe de l'encodeur, et à recourir à un corpus parallèle dédié à la traduction plutôt qu'à un modèle de langage générique pour mieux exploiter le BLEU comme métrique.

5.11 Question de synthèse — Partie III

« Dans quelle mesure les architectures récurrentes permettent-elles de modéliser efficacement une séquence réelle, et comment justifier le passage d'un RNN simple vers un LSTM/GRU puis vers un schéma encodeur–décodeur pour une tâche de génération ou de traduction ? »

Les expériences menées sur IMDb montrent que les architectures récurrentes modélisent efficacement une séquence dans la mesure où elles disposent d'un mécanisme suffisant pour conserver l'information

pertinente sur la durée de la séquence. Le RNN simple, limité par la disparition du gradient lors du BPTT, atteint 69,3 % d'accuracy contre 87,6–88,5 % pour LSTM et GRU : le passage aux architectures à portes est donc directement justifié par un gain de performance mesurable, ces portes régulant explicitement ce qui doit être retenu, oublié ou transmis dans l'état caché au fil du temps.

Le passage vers un schéma encodeur–décodeur (Seq2Seq) se justifie quant à lui par la nature de la tâche elle-même : la classification produit une sortie unique à partir d'une séquence d'entrée, tandis que la génération ou la traduction produisent une séquence de sortie de longueur potentiellement différente de l'entrée. L'encodeur condense la séquence source en un état caché (représentation du « sens » global), que le décodeur déroule ensuite token par token pour produire la séquence cible, le teacher forcing facilitant l'apprentissage en lui fournissant les vrais tokens précédents pendant l'entraînement. Les résultats obtenus ici (perplexité élevée, BLEU nul) montrent cependant que cette architecture, si elle est conceptuellement adaptée à la tâche, requiert un entraînement nettement plus long et plus de données pour produire des séquences cohérentes — la qualité du décodage (glouton vs beam search) ne pouvant compenser un modèle de langage sous-entraîné en amont.

6. Discussion scientifique transversale

« Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ? »

Les trois parties de ce projet illustrent un même principe directeur : la performance d'un modèle de deep learning dépend moins de sa puissance brute (nombre de paramètres) que de l'adéquation entre son biais inductif et la structure géométrique des données qu'il traite.

6.1 Données tabulaires — absence de structure géométrique exploitable

Sur Heart Disease, les variables n'ont pas d'ordre ni de proximité spatiale privilégiée : un MLP, dépourvu de tout biais inductif spécifique, peut en théorie approximer n'importe quelle fonction des features et constitue donc un choix raisonnable, ce que confirment les accuracy de 84 à 92 % obtenues. L'absence de structure à exploiter rend ici les couches denses suffisantes, sans qu'une architecture plus spécialisée n'apporte d'avantage théorique.

6.2 Données image — structure spatiale locale et hiérarchique

Sur CIFAR-10, la proximité spatiale des pixels porte une information sémantique forte (contours, textures, formes), que le CNN exploite via la localité et le partage des poids. Les résultats expérimentaux le confirment sans ambiguïté : le CNN (73,86 %) surpasse de 25 points le MLP ($\approx 47\%$) à budget d'entraînement comparable, et la visualisation des feature maps illustre concrètement la construction hiérarchique de représentations, des contours bas niveau vers des concepts plus abstraits en profondeur.

6.3 Données séquentielles — dépendance temporelle et ordre

Sur IMDb, l'ordre des tokens porte le sens (une phrase n'est pas un sac de mots), ce qui impose un traitement séquentiel et la conservation d'un état au fil du temps. Les architectures à portes (GRU, LSTM) surpassent nettement le RNN simple (88,5 % et 87,6 % contre 69,3 %) en réglant le problème de disparition du gradient sur les dépendances longues, tandis que l'architecture encodeur-décodeur permet d'aller au-delà de la classification pour traiter des tâches où entrée et sortie sont elles-mêmes des séquences de longueurs différentes.

6.4 Synthèse

Un même paradigme — l'apprentissage supervisé par descente de gradient et rétropropagation — se décline donc différemment non pas parce que les algorithmes d'optimisation changent, mais parce que la géométrie des données (absence de structure, structure spatiale 2D, structure temporelle 1D) impose un biais inductif différent à l'architecture pour qu'elle apprenne efficacement à partir d'un nombre raisonnable d'exemples : couches denses pour le tabulaire, convolutions locales partagées pour l'image, récurrence à portes (et au besoin schéma encodeur-décodeur) pour la séquence. Les résultats expérimentaux obtenus dans ce projet — sur trois jeux de données réels et avec des protocoles comparables — confirment empiriquement ce principe théorique central du deep learning moderne.

7. Conclusion générale

Ce projet a permis de mettre en œuvre, comparer et analyser de façon critique trois familles de modèles de deep learning sur trois jeux de données réels représentatifs des grandes catégories de données rencontrées en pratique. Les résultats expérimentaux confirment systématiquement la supériorité d'une architecture adaptée à la structure des données : le MLP suffit pour le tabulaire, le CNN est nécessaire pour l'image, et les architectures récurrentes à portes (GRU/LSTM) s'imposent pour le texte séquentiel. La partie générative (Seq2Seq) illustre par ailleurs les limites pratiques d'un entraînement court face à une tâche intrinsèquement difficile, ouvrant la voie à des pistes d'amélioration (attention, entraînement prolongé, corpus dédié).

8. Limites globales et perspectives

- Nombre d'époques restreint (5 à 50 selon les parties) pour des raisons de temps de calcul, limitant la portée de certaines comparaisons, notamment pour le Seq2Seq.
- Absence de validation croisée et de répétition multi-seed, qui auraient renforcé la robustesse statistique des comparaisons (initialisation, architectures).
- CNN sans data augmentation poussée ni architecture profonde (type ResNet), ce qui plafonne l'accuracy sur CIFAR-10 en dessous de l'état de l'art.
- Seq2Seq sans mécanisme d'attention, pénalisant la qualité du décodage sur des séquences longues.
- Perspective : étendre l'entraînement, introduire l'attention dans le Seq2Seq, et tester des architectures hybrides (CNN-LSTM déjà esquissée dans le notebook CNN) pour des données spatio-temporelles.

9. Annexe expérimentale

L'ensemble des courbes d'apprentissage (loss et accuracy par époque), matrices de confusion, visualisations de feature maps, diagrammes d'architecture RNN/LSTM/GRU et Seq2Seq, ainsi que les tableaux comparatifs détaillés sont disponibles dans les notebooks joints à ce rapport :

- Heart_Disease_MLP.ipynb — Partie I (MLP)
- cnn_cifar10.ipynb — Partie II (CNN)
- Imdb_RNN.ipynb — Partie III (RNN/LSTM/GRU/Seq2Seq)

Ces notebooks constituent le code source complet, commenté, et exécutable, conformément aux livrables attendus.